



PROJECT REPORT: Real-Time Driver Assistance, Lane & Vehicle Detection

Team Members:

- Syed Huzaifa Nazim(23-AI-23)
- Muhammad Waqas Ahmed(23-AI-53)
- Muhammad Basit Memon(23-AI-07)

Section: A1

Subject: Computer Vision

Instructor: Sir Hamza Farooqui

Advanced Driver Assistance System (ADAS)

Pipeline: Technical Project Report

1. Project Title & Overview

Project Name: Real-Time Driver Assistance Lane & Vehicle Detection Pipeline

Core Technologies: OpenCV, Ultralytics YOLOv8, NumPy, SciPy

Target Environment: Python 3.8+

Executive Summary

This project presents an Advanced Driver Assistance System (ADAS) pipeline that blends classical computer vision algorithms with contemporary deep learning frameworks. By processing continuous automotive dashcam footage, the pipeline implements real-time visual safety overlays including vehicle multi-class tracking, automated lane tracking, geometric lane departure monitoring, relative proximity warning detection, and localized traffic flow indexing.

The structural goal of this project is twofold: achieving optimized throughput frames-per-second (FPS) necessary for real-world deployment, and serving as a mathematically transparent educational baseline for industrial computer vision methodologies.

2. Project Architecture & Repository Structure

The software is decoupled across modular execution scripts, keeping diagnostic validation separate from core execution environments.

1.	main.py	# Main pipeline processing video frames & rendering HUD overlays
2.	evaluate.py	# Automated pipeline quality & metric evaluator (50-frame sample)
3.	sobel_manual.py	# Educational script computing manual 2D matrix convolutions
4.	gradient_maps/	# Generated edge maps from raw discrete frames
5.	frame_0_gradient.jpg	
6.	frame_1000_gradient.jpg	
7.	yolov8n.pt	# Deep learning network weights (YOLOv8 Nano variant)
8.	README.md	# Markdown architecture blueprint
9.	LICENSE	# Open-source licensing parameters

3. Mathematical Foundations & Methodology

3.1 Lane Extraction & Geometry Processing

The lane detection component leverages classical digital image processing to filter relevant geometric shapes from video frames:

Grayscale Reduction: Simplifies raw 3-channel RGB matrix feeds into single-channel luminance topologies via discrete channel transformations: $Y \leftarrow 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$

Gaussian Filtering: Suppresses high-frequency background noise (e.g., pavement cracks, gravel textures) using an explicit 5×5 spatial kernel convolution:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

□ **Canny Structural Isolation:** Applies dual-threshold edge isolation based on spatial brightness gradients. High-intensity changes are connected via hysteresis tracking, discarding disjoint structural features.

□ **Spatial Partitioning (Region of Interest):** To optimize compute cycles and minimize false-positive edge associations, a four-point trapezoidal binary vector mask isolates the lower visual region containing the immediate road bed:

$$\text{Vertices} = [(0.10W, H) \quad (0.45W, 0.60H) \quad (0.55W, 0.60H) \quad (0.90W, H)]$$

Probabilistic Hough Transform: Converts edge-pixels into directional line segments by parameterizing points into sinusoidal accumulation parameters:

$$\rho = x \cos(\theta) + y \sin(\theta)$$

3.2 Deep Learning Object Localisation & Inter-Frame Tracking

To bypass the limitations of handcrafted features under variable ambient lightning, a convolutional deep learning backbone isolates moving objects:

- **Model Topology:** Ultralytics YOLOv8 Nano (yolov8n.pt) is chosen for its efficient compute structure, utilizing a modified Darknet backbone paired with a decoupled anchor-free head.
- **Feature Classification Restrictions:** To isolate operational targets relevant to driving decisions, predictions are restricted to targeted class indices from the COCO dataset:
$$\mathcal{C} = \{2:'car', 3:'motorcycle', 5:'bus', 7:'truck'\}$$
- **Temporal Centroid History Queue:** Object state tracking handles spatial associations across sequential frame steps. Centroids are derived from bounding boxes

$$(x_{\text{mid}} = \frac{x_1 + x_2}{2}, y_{\text{mid}} = \frac{y_1 + y_2}{2})$$

and pushed to a size-limited deque ($N=30$) to preserve movement history.

4. Complete Code Base & Comprehensive Reviews

4.1 Production Execution Layer (main.py)

This script manages video file I/O operations, instantiates deep learning weights, triggers the frame analytical engine, and maps spatial information to a real-time HUD matrix overlay.

```
1. import cv2
2. import numpy as np
3. from ultralytics import YOLO
4. from collections import defaultdict
5.
6. # --- Configuration ---
```

```

7. INPUT_VIDEO = "input_video.mp4"
8. OUTPUT_VIDEO = "output_video.mp4"
9. MODEL_PATH = "yolov8n.pt"
10.
11. # Classes to track: 2=car, 3=motorcycle, 5=bus, 7=truck
12. VEHICLE_CLASSES = [2, 3, 5, 7]
13. PROXIMITY_THRESHOLD_RATIO = 0.15 # Area ratio threshold for "Too Close" warning
14.
15. # Dictionary to store vehicle trajectory histories
16. track_history = defaultdict(lambda: [])
17.
18. def region_of_interest(img, vertices):
19.     mask = np.zeros_like(img)
20.     match_mask_color = 255
21.     cv2.fillPoly(mask, vertices, match_mask_color)
22.     masked_image = cv2.bitwise_and(img, mask)
23.     return masked_image
24.
25. def draw_lanes(img, lines, color=[0, 255, 0], thickness=6):
26.     # Separate lines into left and right lanes
27.     left_lines = []
28.     right_lines = []
29.     left_weights = []
30.     right_weights = []
31.
32.     if lines is None:
33.         return img, "Lane Missing"
34.
35.     for line in lines:
36.         for x1, y1, x2, y2 in line:
37.             if x2 == x1:
38.                 continue
39.             slope = (y2 - y1) / (x2 - x1)
40.             intercept = y1 - slope * x1
41.             length = np.sqrt((y2-y1)**2 + (x2-x1)**2)
42.
43.             if slope < -0.3:
44.                 left_lines.append((slope, intercept))
45.                 left_weights.append(length)
46.             elif slope > 0.3:
47.                 right_lines.append((slope, intercept))
48.                 right_weights.append(length)
49.
50.     lane_img = np.zeros_like(img)
51.     lane_status = "Lane Clear"
52.
53.     y1 = img.shape[0]
54.     y2 = int(y1 * 0.6) # Horizon line
55.
56.     # Left Lane
57.     if len(left_lines) > 0:
58.         left_lane = np.dot(left_weights, left_lines) / np.sum(left_weights)
59.         slope, intercept = left_lane
60.         x1 = int((y1 - intercept) / slope)
61.         x2 = int((y2 - intercept) / slope)
62.         cv2.line(lane_img, (x1, y1), (x2, y2), color, thickness)
63.         if x1 > img.shape[1] // 2: # Left line crossed middle
64.             lane_status = "Lane Departure Warning!"
65.
66.     # Right Lane
67.     if len(right_lines) > 0:
68.         right_lane = np.dot(right_weights, right_lines) / np.sum(right_weights)
69.         slope, intercept = right_lane
70.         x1 = int((y1 - intercept) / slope)
71.         x2 = int((y2 - intercept) / slope)
72.         cv2.line(lane_img, (x1, y1), (x2, y2), color, thickness)
73.         if x1 < img.shape[1] // 2: # Right line crossed middle
74.             lane_status = "Lane Departure Warning!"
75.
76.     img = cv2.addWeighted(img, 1.0, lane_img, 1.0, 0.0)
77.     return img, lane_status
78.

```

```

79. def process_lane_detection(frame):
80.     height, width = frame.shape[:2]
81.     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
82.     blur = cv2.GaussianBlur(gray, (5, 5), 0)
83.     edges = cv2.Canny(blur, 50, 150)
84.
85.     # Define Region of Interest (Trapezoid)
86.     region_of_interest_vertices = [
87.         (int(width * 0.1), height),
88.         (int(width * 0.45), int(height * 0.6)),
89.         (int(width * 0.55), int(height * 0.6)),
90.         (int(width * 0.9), height)
91.     ]
92.
93.     cropped_edges = region_of_interest(edges, np.array([region_of_interest_vertices], np.int32))
94.
95.     # Hough Transform
96.     lines = cv2.HoughLinesP(cropped_edges, rho=2, theta=np.pi/180, threshold=50,
97.                             lines=np.array([]), minLineLength=40, maxLineGap=100)
98.
99.     return draw_lanes(frame, lines)
100.
101. def main():
102.     print("Loading YOLO model...")
103.     model = YOLO(MODEL_PATH)
104.
105.     cap = cv2.VideoCapture(INPUT_VIDEO)
106.     if not cap.isOpened():
107.         print(f"Error opening video file {INPUT_VIDEO}")
108.         return
109.
110.     width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
111.     height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
112.     fps = cap.get(cv2.CAP_PROP_FPS)
113.
114.     fourcc = cv2.VideoWriter_fourcc(*'mp4v')
115.     out = cv2.VideoWriter(OUTPUT_VIDEO, fourcc, fps, (width, height))
116.
117.     frame_area = width * height
118.     vehicle_count = 0
119.     counting_line_y = int(height * 0.7) # Line to cross for counting
120.     counted_ids = set()
121.
122.     speed_estimate = 10 # Base simulated speed
123.     frame_count = 0
124.
125.     print("Starting processing...")
126.     while cap.isOpened():
127.         success, frame = cap.read()
128.         if not success:
129.             break
130.
131.         frame_count += 1
132.
133.         # 1. Lane Detection
134.         frame, lane_status = process_lane_detection(frame)
135.
136.         # 2. Vehicle Detection and Tracking
137.         results = model.track(frame, persist=True, classes=VEHICLE_CLASSES, verbose=False)
138.
139.         too_close_warning = False
140.
141.         if results[0].boxes.id is not None:
142.             boxes = results[0].boxes.xyxy.cpu().numpy()
143.             track_ids = results[0].boxes.id.int().cpu().tolist()
144.             cls = results[0].boxes.cls.int().cpu().tolist()
145.
146.             for box, track_id, cls in zip(boxes, track_ids, cls):
147.                 x1, y1, x2, y2 = map(int, box)
148.                 w, h = x2 - x1, y2 - y1
149.                 area = w * h
150.

```

```

151.         # Check proximity
152.         if area > frame_area * PROXIMITY_THRESHOLD_RATIO:
153.             too_close_warning = True
154.             box_color = (0, 0, 255) # Red for danger
155.         else:
156.             box_color = (255, 0, 0) # Blue for normal
157.
158.         cv2.rectangle(frame, (x1, y1), (x2, y2), box_color, 2)
159.
160.         # Trailing logic
161.         center_x, center_y = (x1 + x2) // 2, (y1 + y2) // 2
162.         track_history[track_id].append((center_x, center_y))
163.         if len(track_history[track_id]) > 30: # Retain last 30 frames
164.             track_history[track_id].pop(0)
165.
166.         points = np.hstack(track_history[track_id]).astype(np.int32).reshape((-1, 1, 2))
167.         cv2.polylines(frame, [points], isClosed=False, color=(0, 255, 255), thickness=2)
168.
169.         # Counting logic
170.         if len(track_history[track_id]) >= 2:
171.             prev_y = track_history[track_id][-2][1]
172.             # If moving downwards across the counting line
173.             if prev_y < counting_line_y and center_y >= counting_line_y:
174.                 if track_id not in counted_ids:
175.                     vehicle_count += 1
176.                     counted_ids.add(track_id)
177.
178.         # 3. HUD Overlay
179.         # Simulate slight speed variation
180.         if frame_count % int(fps) == 0:
181.             speed_estimate += np.random.randint(-2, 3)
182.             speed_estimate = max(0, speed_estimate)
183.
184.         # Draw Counting Line
185.         cv2.line(frame, (0, counting_line_y), (width, counting_line_y), (255, 255, 0), 2)
186.
187.         # HUD Text
188.         cv2.putText(frame, f"Speed: {speed_estimate} km/h", (30, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255,
255), 2)
189.         cv2.putText(frame, f"Vehicles Count: {vehicle_count}", (30, 90), cv2.FONT_HERSHEY_SIMPLEX, 1, (255,
255, 255), 2)
190.
191.         lane_color = (0, 0, 255) if "Warning" in lane_status else (0, 255, 0)
192.         cv2.putText(frame, f"Status: {lane_status}", (30, 130), cv2.FONT_HERSHEY_SIMPLEX, 1, lane_color, 2)
193.
194.         if too_close_warning:
195.             cv2.putText(frame, "WARNING: VEHICLE TOO CLOSE!", (width // 2 - 250, height - 100),
196.                         cv2.FONT_HERSHEY_SIMPLEX, 1.5, (0, 0, 255), 3)
197.
198.         out.write(frame)
199.
200.         cap.release()
201.         out.release()
202.         print("Processing complete. Output saved to", OUTPUT_VIDEO)
203.
204. if __name__ == "__main__":
205.     main()
206.

```

Code Review & Implementation Mechanics:

- Weighted Multi-Line Averaging:** In `draw_lanes()`, split segment vectors are segregated into left/right bins based on strict gradient limits ($|m| > 0.3$). Instead of a standard arithmetic mean, lines are aggregated using length weightings (`np.dot(weights, lines) / sum(weights)`). This ensures long, prominent lane markings heavily influence the geometry calculation over intermittent noise segments.

- **Memory Safety:** Track history lists are size-capped via pop(0) constraints when sizes exceed 30 updates. This stabilizes long-term execution profiles by ensuring the heap footprint stays constant.
- **Dynamic Scale Metrics:** Bounding box calculations use normalized frame dimension coefficients (PROXIMITY_THRESHOLD_RATIO = 0.15) instead of hardcoded pixels. This allows the spatial safety triggers to scale seamlessly if inputs shift between 720p, 1080p, or 4K streams.

4.2 Automated Metrics Validation Engine (evaluate.py)

This script isolates performance validation from video assembly, assessing lane recognition accuracy and tracking density profiles across standard 50-frame sampling blocks.

```

1. import cv2
2. import numpy as np
3. from ultralytics import YOLO
4.
5. INPUT_VIDEO = "input_video.mp4"
6. MODEL_PATH = "yolov8n.pt"
7. VEHICLE_CLASSES = [2, 3, 5, 7]
8.
9. def process_lane_detection_eval(frame):
10.     height, width = frame.shape[:2]
11.     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
12.     blur = cv2.GaussianBlur(gray, (5, 5), 0)
13.     edges = cv2.Canny(blur, 50, 150)
14.
15.     region_of_interest_vertices = [
16.         (int(width * 0.1), height),
17.         (int(width * 0.45), int(height * 0.6)),
18.         (int(width * 0.55), int(height * 0.6)),
19.         (int(width * 0.9), height)
20.     ]
21.
22.     mask = np.zeros_like(edges)
23.     cv2.fillPoly(mask, np.array([region_of_interest_vertices], np.int32), 255)
24.     cropped_edges = cv2.bitwise_and(edges, mask)
25.
26.     lines = cv2.HoughLinesP(cropped_edges, rho=2, theta=np.pi/180, threshold=50,
27.                             lines=np.array([]), minLineLength=40, maxLineGap=100)
28.
29.     return lines is not None and len(lines) > 0
30.
31. def main():
32.     print("Loading YOLO model for evaluation...")
33.     model = YOLO(MODEL_PATH)
34.
35.     cap = cv2.VideoCapture(INPUT_VIDEO)
36.     if not cap.isOpened():
37.         print(f"Error opening video file {INPUT_VIDEO}")
38.         return
39.
40.     frames_to_eval = 50
41.     lane_success_count = 0
42.     total_vehicles_detected = 0
43.
44.     print(f"Evaluating {frames_to_eval} frames...")
45.
46.     for i in range(frames_to_eval):
47.         ret, frame = cap.read()
48.         if not ret:
49.             print(f"Video ended early at frame {i}")
50.             frames_to_eval = i
51.             break
52.

```



```

23.     # Convolve
24.     Gx = convolve2d(image, Kx, mode='same', boundary='symm')
25.     Gy = convolve2d(image, Ky, mode='same', boundary='symm')
26.
27.     # Calculate magnitude
28.     magnitude = np.sqrt(Gx**2 + Gy**2)
29.
30.     # Normalize to 0-255
31.     magnitude = np.clip(magnitude, 0, 255)
32.     magnitude = magnitude.astype(np.uint8)
33.
34.     return magnitude
35.
36. def main():
37.     if not os.path.exists(OUTPUT_DIR):
38.         os.makedirs(OUTPUT_DIR)
39.
40.     cap = cv2.VideoCapture(INPUT_VIDEO)
41.     if not cap.isOpened():
42.         print(f"Error opening video file {INPUT_VIDEO}")
43.         return
44.
45.     total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
46.     if total_frames < 10:
47.         print("Video is too short.")
48.         return
49.
50.     # Sample 10 equidistant frames
51.     sample_indices = np.linspace(0, total_frames - 1, 10, dtype=int)
52.
53.     for i, idx in enumerate(sample_indices):
54.         cap.set(cv2.CAP_PROP_POS_FRAMES, idx)
55.         ret, frame = cap.read()
56.         if not ret:
57.             continue
58.
59.         print(f"Processing frame {idx} for gradient map...")
60.         grad_map = manual_sobel(frame)
61.
62.         output_path = os.path.join(OUTPUT_DIR, f"frame_{idx}_gradient.jpg")
63.         cv2.imwrite(output_path, grad_map)
64.
65.     cap.release()
66.     print(f"Saved 10 gradient maps to {OUTPUT_DIR}/")
67.
68. if __name__ == "__main__":
69.     main()
70.

```

Code Review & Discrete Mathematics:

- **Manual Kernel Convolution:** Implements 3×3 horizontal (K_x) and vertical (K_y) edge detection matrices natively. The mathematical convolution per pixel position is defined as:

$$\circ \quad G_x = I * K_x, \quad G_y = I * K_y$$

- **Boundary Condition Handling:** Uses `boundary='symm'` within `scipy.signal.convolve2d`. This mirrors edge pixels symmetrically at the matrix borders, preventing dark border artifacts along the outer edges of the video frames.
- **Vector Normalization:** Combines directional matrices via Euclidean distance mapping
- ($M = \sqrt{G_x^2 + G_y^2}$)
- then safely maps values to unsigned 8-bit integers using `np.clip(..., 0, 255).astype(np.uint8)`.

5. System Execution Blueprint

5.1 System Prerequisites

Ensure your operational container or runtime environment matches these package versions:

```
pip install opencv-python==4.8.0.76 numpy==1.24.3 scipy==1.10.1 ultralytics==8.0.145
```

5.2 Step-by-Step Deployment Guide

1. **Repository Synchronization:** Clone the project files into your local working environment. Place the target video stream file (input_video.mp4) directly inside the root directory folder.
2. **Environment Initialization:** Boot your terminal environment and run the evaluation script to verify your CUDA driver connections and frame pipeline parameters:

```
python evaluate.py
```

Verify Edge Transformations: Extract the spatial edge maps to confirm the region-of-interest filters are properly aligned with your camera's field of view:

```
python sobel_manual.py
```

Run Main Pipeline: Process the full video file, stream it through the tracking logic, and output the annotated HUD video layer:

```
python main.py
```

6. Project Evaluation, Structural Bottlenecks & Future Upgrades

6.1 Performance and Reliability Analysis

- **Classical Lane Extraction Strengths:** The combination of Gaussian blurring and trapezoidal region masking effectively ignores peripheral distractions like roadside barriers or signs. Length-weighted line averaging also provides stable tracking even when lane lines are briefly obscured by passing vehicles.
- **Deep Learning Tracking Strengths:** The anchor-free YOLOv8 architecture excels at detecting vehicles under changing ambient light conditions where traditional hand-crafted features often fail.

6.2 Structural Bottlenecks

1. **Fixed Camera Assumptions:** The hardcoded region-of-interest vertices ([0.10W, H] ...) assume a centered, fixed-horizon camera layout. If the camera is tilted or shifted off-center, the lane detection system may misinterpret the lane lines.
2. **Perspective Distortions in Distance Metrics:** Triggering proximity alerts based on raw bounding box area (area > frame_area * 0.15) can cause false alarms with larger vehicles like commercial buses or semi-trucks, even when they are at a safe following distance.

6.3 Future Upgrades

- **Inverse Perspective Mapping (Bird's Eye View):** Transforming the road region into an unwarped bird's-eye perspective would allow the system to calculate curve radius and lateral lane deviation in actual meters rather than raw pixel counts.
- **Kalman Filtering for Tracking:** Replacing simple centroid queues with a Kalman Filter would provide smoother object trajectories and maintain reliable vehicle tracking even during temporary occlusions.
- **Dynamic Horizon Estimation:** Using a Hough space intersection model would allow the system to dynamically calculate the horizon line, making the lane detection robust to vehicle pitch changes from acceleration, braking, or uneven roads.

7. Open Source Licensing Parameters

This system is maintained under standard development parameters. The underlying code is open for academic modification, expansion, and benchmarking provided original authorship credit is retained.